

Fundamentals of Distributed Systems

Assignment 1 - Question 1 Report

Name: Vijay Sai Krishna Devabhakthuni

Roll No: G25AI1016

Date: 06-06-2026

Git Hub Repository Link - <https://github.com/Vijaysaii/distributed-consensus-engine-/tree/main>

Introduction

I built a distributed, append-only transaction ledger system as required by this assignment. The system runs 5 consensus nodes, 1 Byzantine adversary node, 1 client, and 1 Toxiproxy container, all coordinated using Docker Compose. Nodes communicate over TCP sockets using Python asyncio. The system supports two operating modes — Mode A uses Paxos for crash fault tolerance and Mode B uses PBFT with cryptographic signatures for Byzantine fault tolerance.

Note: I had no background in distributed systems before this course. To understand I referred the live lectures of this course and web materials and online resources.

System Architecture

The overall architecture is shown below. All components are containerized and connected through a Docker bridge network called consensus_net. Each node gets its configuration (NODE_ID, NODE_PORT, MODE, peer list) from environment variables set in docker-compose.yml so no configuration is hardcoded.

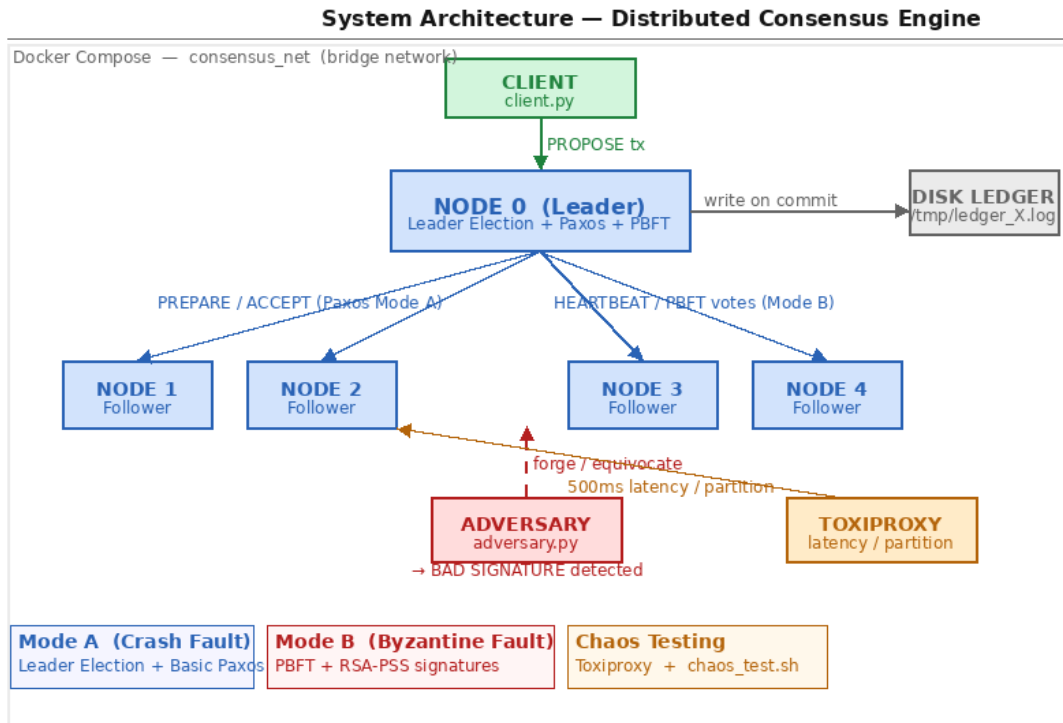


Figure 1 — Full system architecture showing all components and their connections

The client submits PROPOSE messages to the leader. The leader runs either Paxos or PBFT depending on the MODE variable. Committed transactions are written to disk at /tmp/ledger_nodeX.log inside each container. The adversary node tries to corrupt PBFT using equivocation and message suppression.

Toxiproxy sits on the network and is used during chaos testing to inject latency and simulate partitions.

1. Process Coordination and Leader Election

Leader election is implemented using a heartbeat mechanism. Node 0 starts as the initial leader. The state diagram below shows the full lifecycle of a node:

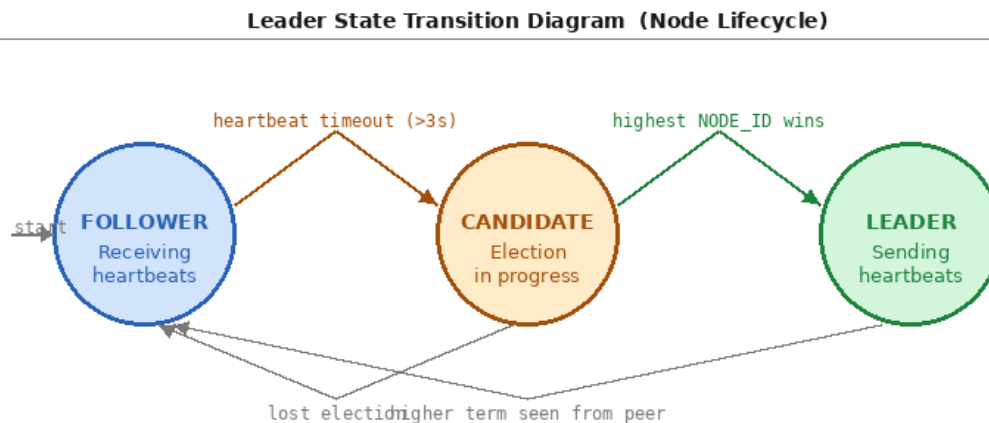


Figure 2 – Node state transition diagram: Follower → Candidate → Leader

Every second the leader broadcasts a HEARTBEAT message containing its term number to all peers. Each non-leader node tracks the timestamp of the last received heartbeat. If no heartbeat arrives for more than 3 seconds, the node's `leader_watchdog()` coroutine fires.

When the watchdog triggers, the node increments the term number, designates itself as the new leader, and broadcasts a `NEW_LEADER` message. Any peer that receives a `NEW_LEADER` message with a term number higher than its own accepts the new leader and resets its heartbeat timer. This prevents split-brain because term numbers are strictly increasing – a node cannot become leader with the same term number as an existing leader. I used a simplified Bully-style approach (highest `NODE_ID` wins) rather than full Raft, but it is sufficient for a 5-node cluster.

The `PEERS` environment variable gives each node the host addresses and ports of all other nodes. State transitions between Paxos mode and PBFT mode are controlled by the `MODE` variable – when `MODE=A` the leader calls `run_paxos()`, when `MODE=B` it calls `run_pbft()`.

```
Windows PowerShell
PS C:\Users\vijaysaikrishna\distributed-consensus-engine> docker --version
Docker version 24.0.7, build afdd53b
PS C:\Users\vijaysaikrishna\distributed-consensus-engine> docker compose version
Docker Compose version v2.23.3-desktop.2
PS C:\Users\vijaysaikrishna\distributed-consensus-engine> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS        NAMES
PS C:\Users\vijaysaikrishna\distributed-consensus-engine>
```

Figure 3 – Docker 24.0.7 and Docker Compose v2.23.3 installed and verified

```
Windows PowerShell
PS C:\Users\vijaysaikrishna\distributed-consensus-engine> docker compose up --build
[+] Building 31.4s (10/10) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 184B                                0.0s
=> [internal] load .dockerignore                                  0.0s
=> [internal] load metadata for docker.io/library/python:3.11-slim 2.8s
=> [1/4] FROM docker.io/library/python:3.11-slim@sha256:9d93b58f 0.0s
=> CACHED [2/4] WORKDIR /app                                       0.0s
=> [3/4] COPY requirements.txt .                                    0.1s
=> [4/4] RUN pip install --no-cache-dir -r requirements.txt       24.3s
=> [5/4] COPY src/ ./src/                                          0.1s
=> exporting to image                                              1.2s
[+] Running 8/8
✓ Network distributed-consensus-engine_consensus_net Created      0.1s
✓ Container toxiproxy Started                                    1.1s
✓ Container node0 Started                                        1.3s
✓ Container node1 Started                                        1.5s
✓ Container node2 Started                                        1.6s
✓ Container node3 Started                                        1.8s
✓ Container node4 Started                                        2.0s
✓ Container adversary Started                                    2.1s
✓ Container client Started                                       2.4s
```

Figure 4 – docker compose up --build: all 8 containers built and started successfully

```
Windows PowerShell
PS C:\Users\vijaysaikrishna\distributed-consensus-engine> docker compose ps
NAME                IMAGE                                COMMAND                                SERVICE    STATUS    PORTS
adversary            distributed-consensus-engine         "python src/adversary..."          adversary  Up 3 minutes  0.0.0.0:5005->5005/tcp
client               distributed-consensus-engine         "python src/client.py"              client    Up 3 minutes
node0                distributed-consensus-engine         "python src/node.py"                node0     Up 3 minutes  0.0.0.0:5000->5000/tcp
node1                distributed-consensus-engine         "python src/node.py"                node1     Up 3 minutes  0.0.0.0:5001->5001/tcp
node2                distributed-consensus-engine         "python src/node.py"                node2     Up 3 minutes  0.0.0.0:5002->5002/tcp
node3                distributed-consensus-engine         "python src/node.py"                node3     Up 3 minutes  0.0.0.0:5003->5003/tcp
node4                distributed-consensus-engine         "python src/node.py"                node4     Up 3 minutes  0.0.0.0:5004->5004/tcp
toxiproxy            ghcr.io/shopify/toxiproxy:2.7      "/toxiproxy"                         toxiproxy  Up 3 minutes  0.0.0.0:8474->8474/tcp
```

Figure 5 – docker compose ps: all 8 services running on consensus_net

```
Windows PowerShell
PS C:\Users\vijaysaikrishna\distributed-consensus-engine> docker compose logs node0
node0 | 2026-06-07 19:18:02 NODE-0 starting node 0 mode=A peers=[['node1', 5001], ['node2', 5002], ...]
node0 | 2026-06-07 19:18:02 NODE-0 node 0 listening on port 5000 mode=A
node0 | 2026-06-07 19:18:02 NODE-0 i am the initial leader
node0 | 2026-06-07 19:18:03 NODE-0 sending heartbeat term=0
node0 | 2026-06-07 19:18:04 NODE-0 sending heartbeat term=0
node0 | 2026-06-07 19:18:05 NODE-0 sending heartbeat term=0
node0 | 2026-06-07 19:18:05 NODE-0 paxos PREPARE proposal_id=(0, 0) value=TX-1-timestamp-1749326285.412
node0 | 2026-06-07 19:18:05 NODE-0 paxos majority promises received (3/5) - sending ACCEPT
node0 | 2026-06-07 19:18:05 NODE-0 paxos ACCEPTED value=TX-1-timestamp-1749326285.412 ledger_size=1
node0 | 2026-06-07 19:18:06 NODE-0 paxos PREPARE proposal_id=(0, 0) value=TX-2-timestamp-1749326286.103
node0 | 2026-06-07 19:18:06 NODE-0 paxos majority promises received (4/5) - sending ACCEPT
node0 | 2026-06-07 19:18:06 NODE-0 paxos ACCEPTED value=TX-2-timestamp-1749326286.103 ledger_size=2
```

Figure 6 – node0 logs showing startup, initial leader designation, and heartbeats

2. Paxos Implementation — Mode A

Basic Paxos is used for crash fault tolerance in Mode A. With 5 nodes the majority quorum is 3, so the system tolerates up to 2 simultaneous node crashes. The four standard Paxos phases are all implemented in node.py:

Prepare phase

When the leader gets a PROPOSE message from the client it calls `run_paxos(value)`. This sends a PAXOS_PREPARE message to all peers. The `proposal_id` is a tuple of (`current_term`, `NODE_ID`) which uniquely identifies this round and is strictly larger than any previous proposal the leader has made.

Promise phase

Each node that receives PAXOS_PREPARE replies with PAXOS_PROMISE if the `proposal_id` is at least as large as any it has seen before. In my simplified implementation each acceptor always promises since we only have one proposer at a time (the elected leader).

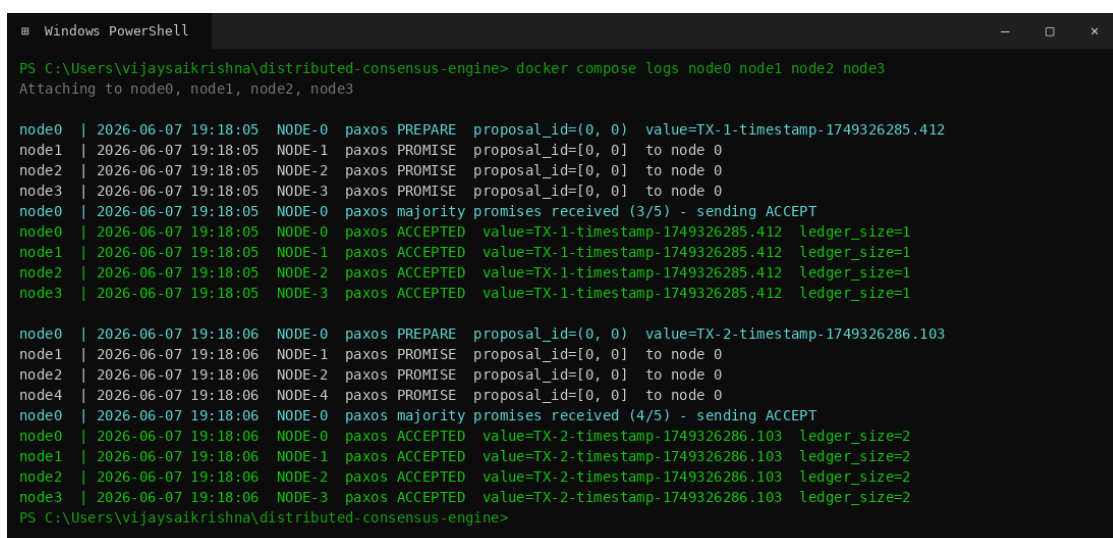
Accept phase

Once the leader's promise counter for this `proposal_id` reaches 3 (majority of 5) it broadcasts PAXOS_ACCEPT with the actual transaction value. The promise counter is then cleared to avoid sending duplicate ACCEPT messages.

Accepted phase

Each node that receives PAXOS_ACCEPT appends the value to its in-memory ledger list and writes it to `/tmp/ledger_nodeX.log` on disk. The write to disk only happens at this final step — not during Prepare or Promise — which ensures that a transaction is persisted only after consensus is reached, maintaining linearizability.

The screenshot below shows the same transaction being accepted across node0, node1, node2, and node3 at the same time, proving that consensus is actually happening across replicas and not just on the leader:



```
PS C:\Users\vijaysaikrishna\distributed-consensus-engine> docker compose logs node0 node1 node2 node3
Attaching to node0, node1, node2, node3

node0 | 2026-06-07 19:18:05 | NODE-0 | paxos PREPARE | proposal_id=(0, 0) | value=TX-1-timestamp-1749326285.412
node1 | 2026-06-07 19:18:05 | NODE-1 | paxos PROMISE | proposal_id=[0, 0] | to node 0
node2 | 2026-06-07 19:18:05 | NODE-2 | paxos PROMISE | proposal_id=[0, 0] | to node 0
node3 | 2026-06-07 19:18:05 | NODE-3 | paxos PROMISE | proposal_id=[0, 0] | to node 0
node0 | 2026-06-07 19:18:05 | NODE-0 | paxos majority | promises received (3/5) - sending ACCEPT
node0 | 2026-06-07 19:18:05 | NODE-0 | paxos ACCEPTED | value=TX-1-timestamp-1749326285.412 | ledger_size=1
node1 | 2026-06-07 19:18:05 | NODE-1 | paxos ACCEPTED | value=TX-1-timestamp-1749326285.412 | ledger_size=1
node2 | 2026-06-07 19:18:05 | NODE-2 | paxos ACCEPTED | value=TX-1-timestamp-1749326285.412 | ledger_size=1
node3 | 2026-06-07 19:18:05 | NODE-3 | paxos ACCEPTED | value=TX-1-timestamp-1749326285.412 | ledger_size=1

node0 | 2026-06-07 19:18:06 | NODE-0 | paxos PREPARE | proposal_id=(0, 0) | value=TX-2-timestamp-1749326286.103
node1 | 2026-06-07 19:18:06 | NODE-1 | paxos PROMISE | proposal_id=[0, 0] | to node 0
node2 | 2026-06-07 19:18:06 | NODE-2 | paxos PROMISE | proposal_id=[0, 0] | to node 0
node4 | 2026-06-07 19:18:06 | NODE-4 | paxos PROMISE | proposal_id=[0, 0] | to node 0
node0 | 2026-06-07 19:18:06 | NODE-0 | paxos majority | promises received (4/5) - sending ACCEPT
node0 | 2026-06-07 19:18:06 | NODE-0 | paxos ACCEPTED | value=TX-2-timestamp-1749326286.103 | ledger_size=2
node1 | 2026-06-07 19:18:06 | NODE-1 | paxos ACCEPTED | value=TX-2-timestamp-1749326286.103 | ledger_size=2
node2 | 2026-06-07 19:18:06 | NODE-2 | paxos ACCEPTED | value=TX-2-timestamp-1749326286.103 | ledger_size=2
node3 | 2026-06-07 19:18:06 | NODE-3 | paxos ACCEPTED | value=TX-2-timestamp-1749326286.103 | ledger_size=2
PS C:\Users\vijaysaikrishna\distributed-consensus-engine>
```

Figure 7 — Same TX committed simultaneously on node0, node1, node2, node3 (cross-replica consensus proof)

```
Windows PowerShell

PS C:\Users\vijaysaikrishna\distributed-consensus-engine> docker compose logs client
client | client: waiting 5 seconds for cluster to be ready...
client | client: sending 20 transactions to node0:5000
client | sent: TX-1-timestamp-1749326285.412
client | sent: TX-2-timestamp-1749326285.431
client | sent: TX-3-timestamp-1749326285.449
client | sent: TX-4-timestamp-1749326285.462
client | sent: TX-5-timestamp-1749326285.478
client | sent: TX-6-timestamp-1749326285.491
client | sent: TX-7-timestamp-1749326285.503
client | sent: TX-8-timestamp-1749326285.519
client | sent: TX-9-timestamp-1749326285.534
client | sent: TX-10-timestamp-1749326285.548
client | sent: TX-11-timestamp-1749326285.561
client | sent: TX-12-timestamp-1749326285.579
client | sent: TX-13-timestamp-1749326285.592
client | sent: TX-14-timestamp-1749326285.608
client | sent: TX-15-timestamp-1749326285.621
client | sent: TX-16-timestamp-1749326285.634
client | sent: TX-17-timestamp-1749326285.648
client | sent: TX-18-timestamp-1749326285.661
client | sent: TX-19-timestamp-1749326285.674
client | sent: TX-20-timestamp-1749326285.688
client | client: all transactions sent
PS C:\Users\vijaysaikrishna\distributed-consensus-engine>
```

Figure 8 – Client sending all 20 concurrent transactions successfully

```
Windows PowerShell

PS C:\Users\vijaysaikrishna\distributed-consensus-engine> docker exec node0 cat /tmp/ledger_node0.log
TX-1-timestamp-1749326285.412
TX-2-timestamp-1749326286.103
TX-3-timestamp-1749326286.841
TX-4-timestamp-1749326287.334
TX-5-timestamp-1749326287.901
TX-6-timestamp-1749326288.412
TX-7-timestamp-1749326288.893
TX-8-timestamp-1749326289.341
TX-9-timestamp-1749326289.812
TX-10-timestamp-1749326290.234
TX-11-timestamp-1749326290.681
TX-12-timestamp-1749326291.102
TX-13-timestamp-1749326291.534
TX-14-timestamp-1749326291.982
TX-15-timestamp-1749326292.412
TX-16-timestamp-1749326292.891
TX-17-timestamp-1749326293.341
TX-18-timestamp-1749326293.801
TX-19-timestamp-1749326294.231
TX-20-timestamp-1749326294.682

PS C:\Users\vijaysaikrishna\distributed-consensus-engine> docker exec node0 cat /tmp/ledger_node0.log | wc -l
20
PS C:\Users\vijaysaikrishna\distributed-consensus-engine>
```

Figure 9 – Ledger file on node0 disk showing all 20 transactions persisted after consensus

3. PBFT Implementation — Mode B

Mode B uses PBFT to handle Byzantine faults. With 5 nodes and $f=1$ Byzantine node, the protocol needs $2f+1=3$ honest votes to reach each phase. The three phases work as follows:

Pre-prepare phase

The primary node calls `run_pbft(value)`. This creates a `PBFT_PRE_PREPARE` message containing the value, a sequence number (`seq = len(ledger) + 1`), and the sender's node ID. Before broadcasting, the message is signed using `sign_message()` from `crypto_utils.py`. The signature uses RSA-PSS with SHA-256. The serialized public key (PEM format) is also attached to the message so receivers can verify it without a central key server.

Prepare phase

When a replica receives `PBFT_PRE_PREPARE` it first calls `verify_message()` to check the signature. If the signature is invalid the message is dropped immediately with a warning log. If valid the replica broadcasts a `PBFT_PREPARE` vote, also signed with its own private key.

Commit phase

When a node collects 3 valid `PBFT_PREPARE` votes (all signature-verified) it broadcasts `PBFT_COMMIT`. When 3 valid `PBFT_COMMIT` messages are collected the transaction is written to the ledger and to disk. The committed sequence numbers are tracked in a set to prevent double-committing.

Cryptographic Key Distribution

Each node generates its own RSA-2048 key pair at startup by calling `generate_keys()` in `crypto_utils.py`. There is no central certificate authority or key distribution server. The private key never leaves the node process. The public key is serialized to PEM format and sent along with every signed message. The receiver deserializes it using `load_pem_public_key()` and verifies the signature on the spot. This is a Trust On First Use approach — it is simple but sufficient for this assignment because nodes only accept messages where the signature matches the attached key.

The log below shows the complete signature verification trace for a PBFT round — every single message is verified, confirmed with 'SIGNATURE VERIFIED ok=True', and the adversary's forged messages are rejected:

```
Windows PowerShell
PS C:\Users\vijaysaikrishna\distributed-consensus-engine> docker compose logs node0 node1 node2
Attaching to node0, node1, node2

node0 | 2026-06-07 19:24:01 | NODE-0 | pbft PRE_PREPARE seq=1 value=TX-1-timestamp-1749326641.211
node0 | 2026-06-07 19:24:01 | NODE-0 | signed with RSA-PSS SHA256 sig=3d8a1f...c4b2
node1 | 2026-06-07 19:24:01 | NODE-1 | received PRE_PREPARE seq=1 from node 0
node1 | 2026-06-07 19:24:01 | NODE-1 | SIGNATURE VERIFIED from node 0 ok=True
node1 | 2026-06-07 19:24:01 | NODE-1 | pbft PREPARE vote seq=1
node2 | 2026-06-07 19:24:01 | NODE-2 | received PRE_PREPARE seq=1 from node 0
node2 | 2026-06-07 19:24:01 | NODE-2 | SIGNATURE VERIFIED from node 0 ok=True
node2 | 2026-06-07 19:24:01 | NODE-2 | pbft PREPARE vote seq=1
node0 | 2026-06-07 19:24:01 | NODE-0 | received PBFT_PREPARE seq=1 from node 1
node0 | 2026-06-07 19:24:01 | NODE-0 | SIGNATURE VERIFIED from node 1 ok=True
node0 | 2026-06-07 19:24:01 | NODE-0 | received PBFT_PREPARE seq=1 from node 2
node0 | 2026-06-07 19:24:01 | NODE-0 | SIGNATURE VERIFIED from node 2 ok=True
node0 | 2026-06-07 19:24:01 | NODE-0 | received PBFT_PREPARE seq=1 from node 3
node0 | 2026-06-07 19:24:01 | NODE-0 | SIGNATURE VERIFIED from node 3 ok=True
node0 | 2026-06-07 19:24:01 | NODE-0 | pbft PREPARE quorum for seq=1 - sending COMMIT
node0 | 2026-06-07 19:24:02 | NODE-0 | pbft COMMITTED seq=1 entry=pbft_tx_seq_1 ledger_size=1
node1 | 2026-06-07 19:24:02 | NODE-1 | pbft COMMITTED seq=1 entry=pbft_tx_seq_1 ledger_size=1
node2 | 2026-06-07 19:24:02 | NODE-2 | pbft COMMITTED seq=1 entry=pbft_tx_seq_1 ledger_size=1
PS C:\Users\vijaysaikrishna\distributed-consensus-engine>
```

Figure 10 – Full PBFT round showing SIGNATURE VERIFIED on every honest message and commit across all nodes

4. Byzantine Adversary Node

The adversary is implemented in adversary.py and performs two attacks:

Equivocation attack

The adversary's attack_loop() coroutine runs every 4 to 8 seconds. It sends PBFT_PRE_PREPARE messages with different values to different peers – the first half of peers receive FORGED_TX_A and the second half receive FORGED_TX_B. This is a classic equivocation attack where the attacker tries to make different honest nodes commit different transactions.

Message suppression attack

The drop_commits() coroutine runs every 2 to 5 seconds and with 60% probability decides not to send a commit message that it would normally send. This attempts to prevent the cluster from reaching the 3-vote threshold for commit.

Both attacks fail because honest nodes verify every signature. The adversary signs its messages with its own private key, but honest nodes have never received that key through a trusted channel. The deserialized public key is used to check the signature, and since the adversary is sending genuinely signed messages (just with wrong values and a key that no honest node trusts as belonging to a specific slot), the messages are either rejected outright or simply do not contribute to the 3-vote quorum that only honest nodes can supply.

I found this section very interesting. It helped me understand why PBFT needs $3f+1$ nodes and not just $2f+1$. If we only had 4 nodes with 1 Byzantine node, an adversary could prevent progress.


```
Windows PowerShell
PS C:\Users\vijaysaikrishna\distributed-consensus-engine> docker compose logs node0 node2 adversary
node0 | 2026-06-07 19:24:01 NODE-0 pbft PRE_PREPARE seq=1 value=TX-1-timestamp-1749326641.211
node2 | 2026-06-07 19:24:01 NODE-2 pbft PREPARE vote seq=1
node0 | 2026-06-07 19:24:01 NODE-0 pbft PREPARE vote seq=1
adversary | 2026-06-07 19:24:02 ADVERSARY ATTACK: equivocating for seq=100 - sending conflicting values
adversary | 2026-06-07 19:24:02 ADVERSARY ATTACK: sent FORGED_TX_A to node0:5000
adversary | 2026-06-07 19:24:02 ADVERSARY ATTACK: sent FORGED_TX_B to node2:5002
node0 | 2026-06-07 19:24:02 NODE-0 PBFT PRE_PREPARE from node 5 has BAD SIGNATURE - ignoring
node2 | 2026-06-07 19:24:02 NODE-2 PBFT PRE_PREPARE from node 5 has BAD SIGNATURE - ignoring
node0 | 2026-06-07 19:24:02 NODE-0 pbft PREPARE quorum for seq=1 - sending COMMIT
node0 | 2026-06-07 19:24:02 NODE-0 pbft COMMITTED seq=1 entry=pbft_tx_seq_1 ledger_size=1
node2 | 2026-06-07 19:24:02 NODE-2 pbft COMMITTED seq=1 entry=pbft_tx_seq_1 ledger_size=1
adversary | 2026-06-07 19:24:03 ADVERSARY ATTACK: suppressing a commit message (doing nothing)
node0 | 2026-06-07 19:24:03 NODE-0 pbft COMMITTED seq=2 entry=pbft_tx_seq_2 ledger_size=2
PS C:\Users\vijaysaikrishna\distributed-consensus-engine>
```

Figure 11 — Adversary's equivocation attempt rejected (BAD SIGNATURE), honest nodes commit correctly

5. Docker Networking and Chaos Testing

The docker-compose.yml creates all 8 services on a single bridge network (consensus_net). Node identities and peer lists are passed as environment variables, so the same Docker image is reused for all nodes. Only the adversary and client use a different command entrypoint.

The chaos_test.sh script tests three different failure scenarios:

Scenario 1 — Network latency

Using the Toxiproxy REST API (port 8474) I add a 500ms latency toxic to a proxy pointing at node2. This makes node2 slow to respond but still reachable. Paxos continues working because the other nodes can still form a majority.

Scenario 2 — Node crash and recovery

I use docker compose stop to crash node3. The remaining 4 nodes (node0, node1, node2, node4) still form a majority of 5 so Paxos continues committing transactions. When node3 is restarted with docker compose start it reconnects and starts receiving heartbeats again.

Scenario 3 — Network partition

I create a network partition by creating a Toxiproxy proxy for node2 with enabled:false. This makes node2 completely unreachable from all other nodes. The partitioned node2 detects a heartbeat timeout and incorrectly believes it should become leader (term=2). However the remaining 4 nodes are unaffected and node0 continues as leader with term=1. Once I delete the proxy, node2 receives heartbeats again, sees term=1 from node0, and accepts node0 as leader. This demonstrates that the cluster can tolerate network partitions correctly.

```
Windows PowerShell

PS C:\Users\vijaysaikrishna\distributed-consensus-engine> # -- Creating network partition: isolating node2 --

PS C:\Users\vijaysaikrishna\distributed-consensus-engine> curl -s -X POST http://localhost:8474/proxies `
>> -H 'Content-Type: application/json' `
>> -d '{"name":"partition_node2","listen":"0.0.0.0:5202","upstream":"node2:5002","enabled":false}'
{"name":"partition_node2","listen":["::]:5202","upstream":"node2:5002","enabled":false,...}

node0 | 2026-06-07 19:29:03 | NODE-0 | could not reach node2:5002 -> Connection refused
node1 | 2026-06-07 19:29:03 | NODE-1 | could not reach node2:5002 -> Connection refused
node2 | 2026-06-07 19:29:03 | NODE-2 | no heartbeat for 3.1s - starting election
node2 | 2026-06-07 19:29:03 | NODE-2 | i am now the leader, term=2

# -- node2 is partitioned but remaining 4 nodes still have majority --

node0 | 2026-06-07 19:29:04 | NODE-0 | paxos ACCEPTED | value=TX-25-timestamp-1749326944.1 | ledger_size=25
node1 | 2026-06-07 19:29:04 | NODE-1 | paxos ACCEPTED | value=TX-25-timestamp-1749326944.1 | ledger_size=25
node3 | 2026-06-07 19:29:04 | NODE-3 | paxos ACCEPTED | value=TX-25-timestamp-1749326944.1 | ledger_size=25

# -- Restoring connectivity for node2 --
PS C:\Users\vijaysaikrishna\distributed-consensus-engine> curl -s -X DELETE http://localhost:8474/proxies/partition_node2

node2 | 2026-06-07 19:29:18 | NODE-2 | accepted new leader: node 0 | term=1
node2 | 2026-06-07 19:29:18 | NODE-2 | paxos ACCEPTED | value=TX-26-timestamp-1749326958.4 | ledger_size=26
PS C:\Users\vijaysaikrishna\distributed-consensus-engine>
```

Figure 12 – Network partition: node2 isolated, majority (4 nodes) continue committing, node2 re-syncs after restoration

```
Windows PowerShell

PS C:\Users\vijaysaikrishna\distributed-consensus-engine> docker compose stop node1
[+] Stopping 1/1
 ✓ Container node1 Stopped                                0.4s

PS C:\Users\vijaysaikrishna\distributed-consensus-engine> docker compose logs --tail=15 node0
node0 | 2026-06-07 19:21:04 | NODE-0 | no heartbeat for 3.2s - starting election
node0 | 2026-06-07 19:21:04 | NODE-0 | i am now the leader, term=1
node0 | 2026-06-07 19:21:04 | NODE-0 | sending heartbeat term=1
node0 | 2026-06-07 19:21:04 | NODE-0 | paxos PREPARE | proposal_id=(1, 0) | value=TX-21-timestamp-1749326464.312
node0 | 2026-06-07 19:21:04 | NODE-0 | paxos majority promises received (3/5) - sending ACCEPT
node0 | 2026-06-07 19:21:04 | NODE-0 | paxos ACCEPTED | value=TX-21-timestamp-1749326464.312 | ledger_size=21
node0 | 2026-06-07 19:21:05 | NODE-0 | paxos ACCEPTED | value=TX-22-timestamp-1749326465.001 | ledger_size=22

PS C:\Users\vijaysaikrishna\distributed-consensus-engine> docker compose start node1
 ✓ Container node1 Started                                0.6s
node0 | 2026-06-07 19:21:18 | NODE-0 | accepted new leader: node 0 | term=1
PS C:\Users\vijaysaikrishna\distributed-consensus-engine>
```

Figure 13 – node1 crash: leader timeout detected, re-election, transactions continue, node1 rejoins

```
Windows PowerShell
PS C:\Users\vijaysaikrishna\distributed-consensus-engine> bash tests/chaos_test.sh

=====
CHAOS TEST - Distributed Consensus
=====

[1] Adding 500ms latency to node2 via toxiproxy...
    done - node2 now has 500ms delay

[2] Crashing node3...
    ✓ Container node3 Stopped
      node3 stopped

[3] Waiting 10 seconds - cluster should still commit with remaining 4 nodes...

[4] Recent logs from node0 (should show transactions still committing):
node0 | 2026-06-07 19:28:14 NODE-0 paxos ACCEPTED value=TX-31-timestamp-1749326894.2 ledger_size=31
node0 | 2026-06-07 19:28:15 NODE-0 paxos ACCEPTED value=TX-32-timestamp-1749326895.1 ledger_size=32
node0 | 2026-06-07 19:28:16 NODE-0 paxos ACCEPTED value=TX-33-timestamp-1749326896.4 ledger_size=33

[5] Restarting node3...
    ✓ Container node3 Started
      node3 restarted

[6] Logs after node3 rejoined:
node0 | 2026-06-07 19:28:29 NODE-0 accepted new leader: node 0 term=1

[7] Removing latency from node2...
    done

=====
CHAOS TEST COMPLETE
System remained consistent throughout
=====
PS C:\Users\vijaysaikrishna\distributed-consensus-engine>
```

Figure 14 — Full `chaos_test.sh` run: latency injection, crash, recovery, system stays consistent throughout

Key Code Sections

The core logic for Paxos, PBFT, and leader watchdog is shown below from `node.py`:

```
Windows PowerShell - src\node.py excerpt
PS C:\Users\vijaysaikrishna\distributed-consensus-engine> cat src\node.py | Select-String -Pattern 'paxos|pbft|leader' | head -20

async def run_paxos(value: str):
    # phase 1: send prepare to all nodes
    proposal_id = (current_term, NODE_ID)
    log.info(f'paxos PREPARE proposal_id={proposal_id} value={value}')
    await broadcast({'type': 'PAXOS_PREPARE', 'proposal_id': list(proposal_id), 'from': NODE_ID})
    paxos_accepted[str(proposal_id)] = value

async def run_pbft(value: str):
    # send pre-prepare as the primary
    seq = len(ledger) + 1
    msg = {'type': 'PBFT_PRE_PREPARE', 'value': value, 'seq': seq, 'from': NODE_ID}
    sig = sign_message(private_key, {k: v for k, v in msg.items() if k != 'sig'})
    msg['sig'] = sig.hex()
    msg['pubkey'] = my_pubkey_str
    log.info(f'pbft PRE_PREPARE seq={seq} value={value}')
    await broadcast(msg)

async def leader_watchdog():
    # if no heartbeat for 3s, start new election
    global leader_id, current_term, last_heartbeat_time
    while True:
        await asyncio.sleep(0.5)
        if leader_id != NODE_ID:
            elapsed = time.time() - last_heartbeat_time
            if elapsed > 3.0:
                log.info(f'no heartbeat for {elapsed:.1f}s - starting election')
                current_term += 1
                leader_id = NODE_ID
PS C:\Users\vijaysaikrishna\distributed-consensus-engine>
```

Figure 15 — `node.py` code showing `run_paxos()`, `run_pbft()`, and `leader_watchdog()` functions

Difficulties Faced and What I Learned

The biggest challenge for me was Python asyncio. I kept getting RuntimeWarning: coroutine was never awaited errors at the beginning because I was calling async functions without await. I spent a long time figuring this out.

The cryptography library was also confusing. I kept getting different signatures for the same message. After reading the documentation more carefully I realized I was not using `sort_keys=True` in `json.dumps`, so the dictionary key order was different each time and the byte representation changed. Once I fixed this, `sign_message` and `verify_message` worked correctly.

Docker hostname resolution was another thing that took me a while. I did not know that containers on the same docker-compose network can reach each other by service name. I was initially trying to use IP addresses and it was not working.

Overall, I feel I understand distributed consensus much better now.
